Control de Cambios e Informe de Incorporación de Patrones de Diseño

Patrones creacionales y estructurales · Proyecto RUMBANG



UNIVERSIDAD
Popular del cesar

HECTOR ARTURO GUERRA PEREZ
MAIKEL ALEXIS PAYARES DOMINGUEZ

Especialización en Ingeniería de Software · Patrones de Diseño de Software
Facultad de Ingeniería y Tecnología
Valledupar 2026-2

Control de Cambios — Incorporación de Patrones (Entrega 3)

Asignatura: Patrones de Diseño de Software · Proyecto: RUMBANG (Godot 4.7 / GDScript) Prefijo **PE** = Patrón Estructural/Creacional. Se incluyen los 5 creacionales y 4 estructurales (mínimo exigido: 3).

ID	Clase(s) Modificada(s)	Patrón Aplicado (Creacional / Estructural)	Problema Previo (E2 o código refactorizado)	Mejora Obtenida con el Patrón
PE-01	**Nuevas:** `EnemyCreator`, `BossCreator`, `NecromancerBossCreator` (`scripts/patterns/factory_method/`). **Modificadas:** `main.gd`, `wave_manager.gd`. **Eliminada:** `factory_method_pattern.gd` (`EnemyFactory`)	**Factory Method** (Creacional)	`EnemyFactory.create_enemy(tipo: String)` era una fábrica estática con `match` sobre strings y `preload` de rutas: no había jerarquía Creator/ConcreteCreator, el tipo `"boss"` nunca se usaba y cada jefe nuevo obligaba a editar la fábrica (viola OCP).	Un jefe nuevo = una subclase de `EnemyCreator` que sobreescribe `_create_product()`; los clientes dependen del Creator, no de rutas ni de strings. La configuración común queda en una sola operación (`create_enemy()`).
PE-02	`PlatformFactory`, `MobilePlatformFactory`, `DesktopPlatformFactory` (`abstract_factory_pattern.gd`); cliente `game_start_facade.gd`	**Abstract Factory** (Creacional) — ya existente, auditado y documentado	Condicionales `if OS.get_name() in ["Android","iOS"]` dispersos en `main.gd` mezclaban reglas de negocio con detalles de plataforma.	Una única decisión (`PlatformFactory.get_factory()`) produce la familia coherente de configuración (entorno, viewport, luz, controles, mapa) para móvil o escritorio. Añadir una plataforma = una fábrica concreta.
PE-03	`WeaponBuilder` (`builder_pattern.gd`); **nueva:** `WeaponDirector` (`builder_director.gd`); **modificada:** `weapon_data.gd`	**Builder** (Creacional) — se completa con el rol **Director**	Las armas se construían con un constructor de 19 parámetros / diccionarios grandes; con el Builder cada arma repetía a mano el orden de pasos (`set_combat` + `set_projectile`), con riesgo de olvidar uno.	`WeaponDirector.construct_melee()` / `construct_ranged()` encapsulan las recetas; 6 de las 11 armas se declaran en una línea. Valores finales idénticos (verificado).
PE-04	**Nuevas:** `WeaponPrototypeRegistry` (`prototype/`). **Modificada:** `WeaponData` (`clone()`, `get_weapon_table()`)	**Prototype** (Creacional)	`_apply_balance()` mutaba las fichas originales: se perdían los valores base de cada arma y no existía copia independiente de una ficha.	Las fichas del Builder se registran como prototipos intactos y la tabla de juego se obtiene **clonándolos** y balanceando la copia. Se puede pedir un clon sin balance (`clone_of`).
PE-05	`GameState` (`scripts/core/game_state.gd`). **Eliminada:** `singleton_pattern.gd` (`GameManagerSingleton`, código muerto sin ningún uso)	**Singleton** (Creacional)	Existían dos Singleton: `GameState` (en uso, 10 referencias) y `GameManagerSingleton` (duplicado que nadie llamaba), más Autoloads que instancian nodos solo para guardar estado.	Un único Singleton de estado de partida (`GameState.get_instance()`) sin depender del `SceneTree`; se elimina el duplicado y la ambigüedad. Los Autoloads se documentan como Singletons gestionados por el motor.
PE-06	**Nuevas:** `DamageSource`, `BaseWeaponDamage`, `DamageDecorator`, `BerserkDamageDecorator`, `UpgradeDamageDecorator`, `StatBlockDamageDecorator` (`decorator/`). **Modificada:** `player.gd` (`_weapon_damage`, `_weapon_heavy_damage`)	**Decorator** (Estructural)	`_weapon_damage()` y `_weapon_heavy_damage()` repetían la misma cadena de multiplicadores (berserk × mejoras × stats) escrita a mano; añadir un modificador exigía tocar ambas.	Cada modificador es un decorador apilable; ambas funciones comparten `_damage_chain()`. Un modificador nuevo (p. ej. crítico) = una clase nueva, sin editar las existentes.

ID	Clase(s) Modificada(s)	Patrón Aplicado (Creacional / Estructural)	Problema Previo (E2 o código refactorizado)	Mejora Obtenida con el Patrón
PE-07	**Nuevas:** `EnemyTypeFlyweight`, `EnemyTypeFlyweightFactory` (`flyweight/`). **Modificada:** `enemy_behavior.gd` (6 `apply_config()`)	**Flyweight** (Estructural)	Cada `EnemyData` (cientos simultáneos) escribía 8 valores literales de su tipo en `apply_config()`; los datos del tipo estaban duplicados en 6 clases de comportamiento.	El estado intrínseco (vida, velocidad, alcance, daño, oro, escala) se comparte: solo 6 flyweights en memoria; el estado extrínseco (posición, hp actual, IA) sigue por enemigo. El balance de un tipo se cambia en un único lugar.
PE-08	**Nuevas:** `MovementSource`, `TouchMovementAdapter`, `KeyboardMovementAdapter`, `GamepadMovementAdapter` (`adapter/`). **Modificada:** `player_input_controller.gd`	**Adapter** (Estructural)	`get_movement_vector()` conocía a la vez el autoload `VirtualInput`, las teclas WASD y los ejes del mando (tres APIs mezcladas en un método).	Cada fuente de entrada se adapta a la interfaz `MovementSource`; el controlador solo combina fuentes. Un periférico nuevo = un adaptador nuevo.
PE-09	`GameStartFacade` (`scripts/core/game_start_facade.gd`); cliente `main.gd`	**Facade** (Estructural) — ya existente (Entrega 2)	`Main` orquestaba 4+ subsistemas (UI, entorno, oleadas, botín) directamente.	`Main` habla con una sola fachada (`initialize`, `start_game`, `update_gold`...).

Ajuste complementario (no GoF)

ID	Clase	Problema previo	Mejora
AJ-01	`PoolManager` (Object Pool)	`get_bullet()` recorría el pool en anillo sin comprobar si la bala seguía en vuelo: con más de 50 balas simultáneas reutilizaba una activa.	Busca primero una bala libre; solo si todas están en uso recicla la más antigua (mismo comportamiento de respaldo que antes, sin devolver `null` a los llamadores). Igual para los números flotantes.

Verificación realizada

- Proyecto abre en Godot 4.7.1 sin errores de script (`--headless --editor --quit`).
- La escena de juego (`main_procedural.tscn`) corre 600 fotogramas sin errores de script.
- Prueba automática: 11 armas en la tabla; espada balanceada 10.2 de daño frente al prototipo intacto de 12.0; cadena Decorator = 24.8625 = valor esperado; `get_flyweight()` devuelve la misma instancia; ambos Creators producen su nodo.

Informe de Incorporación de Patrones Creacionales y Estructurales

Proyecto **RUMBANG** (Godot 4.7, GDScript) · Entrega 3 · Punto de partida: código refactorizado con SOLID de la Entrega 2. Trazabilidad completa en el Control de Cambios (sección anterior); los diagramas de clases antes y después aparecen al final de cada patrón.

Cada patrón sigue la estructura de participantes GoF y responde: **qué** es, **cómo** se aplicó, **por qué** (problema previo), beneficios y desventajas, con evidencia de código antes y después.

PE-01 · Factory Method (creacional)

Qué. Define una operación para crear un objeto y delega en las subclases qué clase concreta instanciar. **Participantes → clases.** Creator → `EnemyCreator`; ConcreteCreator → `BossCreator`, `NecromancerBossCreator`; Product → nodo de enemigo (`Node3D`); ConcreteProduct → `boss.tscn`, `necromancer_boss.tscn`. **Por qué.** `EnemyFactory` no era un Factory Method: era una fábrica estática con `match` sobre strings. Violaba OCP y acoplaba a los clientes (`main.gd`, `wave_manager.gd`) a literales como `"necromancer"`.

ANTES (`factory_method_pattern.gd`)

```
static func create_enemy(type: String) -> Node3D:
  var enemy_scene: PackedScene
  match type:
    "boss": enemy_scene = preload("res://scenes/entities/boss.tscn")
    "necromancer": enemy_scene = preload("res://scenes/entities/necromancer_boss.tscn")
    _: push_error("Tipo de enemigo desconocido: " + type); return null
  ...
# cliente:
var boss = EnemyFactory.create_enemy("necromancer")
```

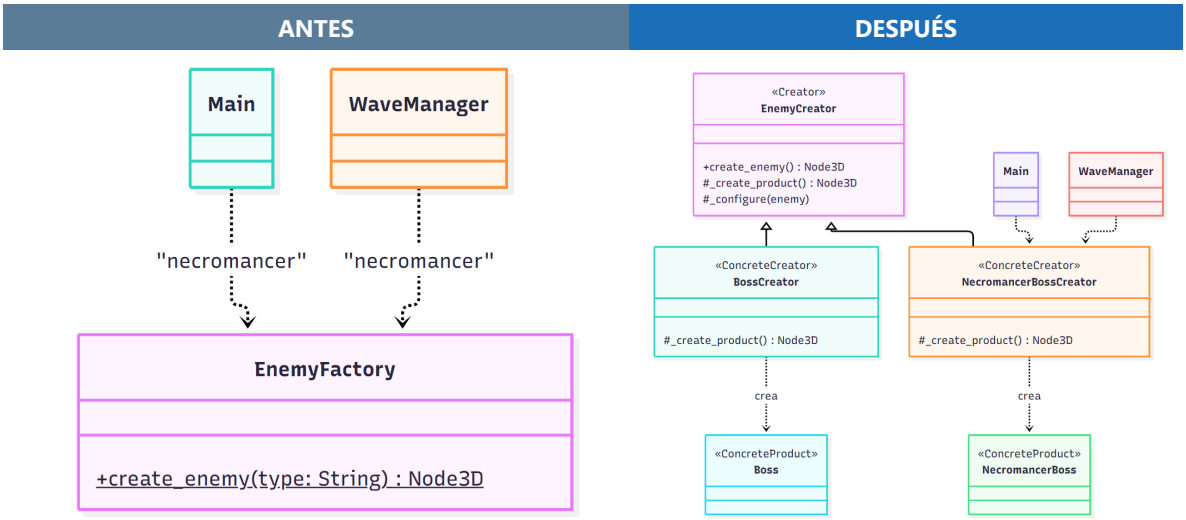
DESPUÉS

```
# enemy_creator.gd (Creator)
func create_enemy() -> Node3D:
  var enemy: Node3D = _create_product()      # factory method
  if enemy == null: ...; return null
  _configure(enemy)
  return enemy
func _create_product() -> Node3D: return null

# necromancer_boss_creator.gd (ConcreteCreator)
func _create_product() -> Node3D:
  return NECROMANCER_SCENE.instantiate() as Node3D
# cliente:
var boss = NecromancerBossCreator.new().create_enemy()
```

Beneficios. OCP real (jefe nuevo = clase nueva); sin strings mágicos; configuración común en un solo punto.
Desventajas. Más archivos/clases para solo dos productos; una subclase por producto.

Diagramas de clases



PE-02 · Abstract Factory (creacional) — auditado

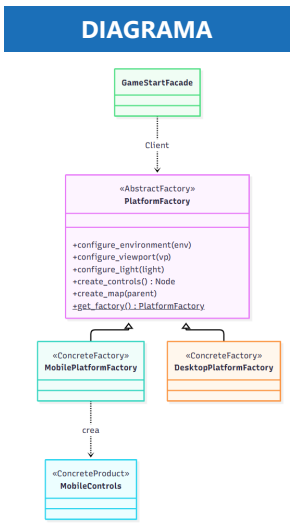
Qué. Interfaz para crear familias de objetos relacionados sin especificar sus clases concretas. **Participantes → clases.** AbstractFactory → `PlatformFactory`; ConcreteFactory → `MobilePlatformFactory`, `DesktopPlatformFactory`; AbstractProduct/ConcreteProduct → configuración de entorno/luz/viewport, controles (`MobileControls`) y mapa; Client → `GameStartFacade._setup_platform()`. **Por qué.** Se reemplazaron los `if OS.get_name() in [...]` esparcidos por una única decisión.

ANTES: condicionales de plataforma en `main.gd`. DESPUÉS

```
var platform_factory: PlatformFactory = PlatformFactory.get_factory()
platform_factory.configure_environment(env)
platform_factory.configure_light(light)
var controls = platform_factory.create_controls()
```

Beneficios. Coherencia de la familia móvil/escritorio; añadir plataforma sin tocar al cliente. **Desventajas / limitación conocida.** Casi todas las operaciones son `configure_*` sobre objetos existentes (solo `create_controls` devuelve un producto), así que es un Abstract Factory parcial; `get_factory()` en la clase base referencia a sus subclases.

Diagramas de clases



PE-03 · Builder + Director (creacional)

Qué. Separa la construcción de un objeto complejo de su representación, paso a paso. **Participantes → clases.** Builder/ConcreteBuilder → `WeaponBuilder`; Director → `WeaponDirector`; Product → `WeaponData`. **Por qué.** El Builder existía pero cada arma repetía el orden de pasos a mano; faltaba el Director.

ANTES

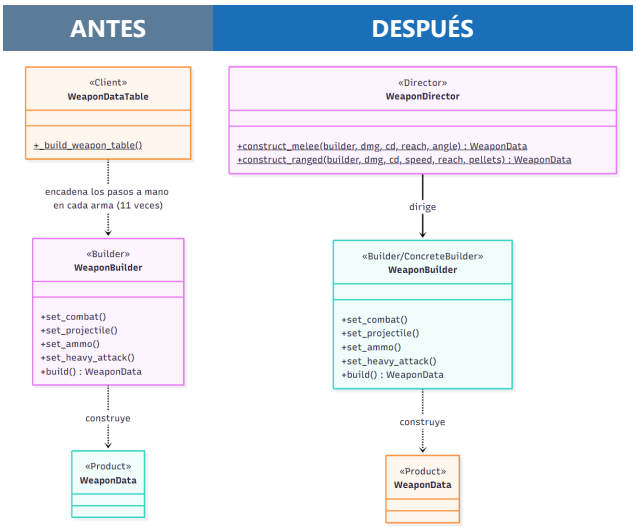
```
WeaponType.SPEAR_LANCE: WeaponBuilder.new(5, "Espalanza de Mikeura", "…", "melee")\
  .set_combat(15.0, 0.50, 5.0)\
  .set_projectile(0.0, 1, 80.0)\
  .build(),
```

DESPUÉS

```
WeaponType.SPEAR_LANCE: WeaponDirector.construct_melee(
  WeaponBuilder.new(5, "Espalanza de Mikeura", "…", "melee"), 15.0, 0.50, 5.0, 80.0),
```

Beneficios. Recetas reutilizables; 6 armas en una línea; valores idénticos (probado). **Desventajas.** Las 5 armas con configuración irregular (escopeta, railgun, bazuca, polvos, serpientes) siguen usando el Builder directo; no hay interfaz `Builder` abstracta porque solo existe un constructor concreto.

Diagramas de clases



PE-04 · Prototype (creacional)

Qué. Crea nuevos objetos copiando un prototipo. **Participantes → clases.** Prototype → `WeaponData` (`clone()`); ConcretePrototype → cada ficha de arma; Client → `WeaponPrototypeRegistry` y `WeaponData.get_weapon_table()`. **Por qué.** `_apply_balance()` mutaba las fichas originales; los valores base se perdían.

ANTES

```
_weapon_table_cache = _build_weapon_table()
_apply_balance(_weapon_table_cache)        # modifica las originales
```

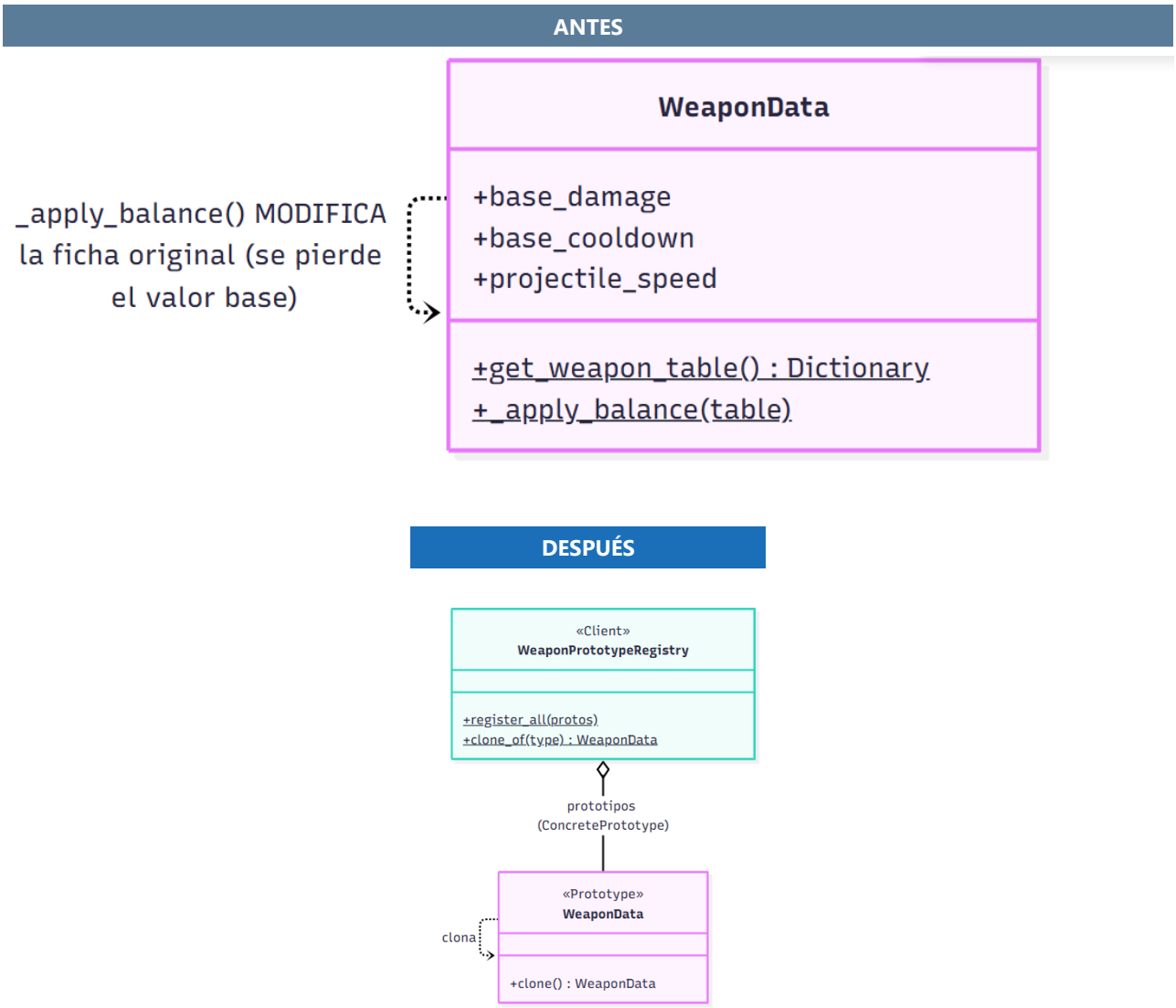
DESPUÉS

```
WeaponPrototypeRegistry.register_all(_build_weapon_table())   # prototipos intactos
for key in WeaponType.values():
  if WeaponPrototypeRegistry.has_prototype(key):
    _weapon_table_cache[key] = WeaponPrototypeRegistry.clone_of(key)  # clon
_apply_balance(_weapon_table_cache)        # solo toca los clones
```

Beneficios. Prototipos inmutables; copia independiente por arma (espada: 12.0 en el prototipo, 10.2 en el clon balanceado). **Desventajas.** `duplicate()` es superficial (suficiente aquí: solo tipos primitivos); dos copias de cada ficha en

memoria.

Diagramas de clases



PE-05 · Singleton (creacional)

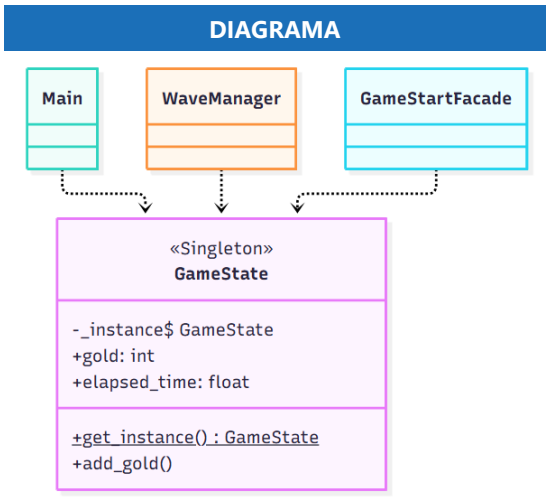
Qué. Garantiza una única instancia con acceso global. **Participantes → clases.** Singleton → `GameState` (`static var _instance`, `get_instance()`). Además los Autoloads de `project.godot` (`GameData`, `AudioManager`, `PoolManager`…) son Singletons gestionados por el motor. **Por qué.** Había un duplicado muerto, `GameManagerSingleton`, sin ninguna referencia; dos Singleton para el mismo propósito confunden el diseño.

ANTES: `GameState` (en uso) + `GameManagerSingleton` (sin usos, con `current_score` y `active_enemies`). DESPUÉS: solo `GameState`.

```
static var _instance: GameState = null
static func get_instance() -> GameState:
  if _instance == null: _instance = GameState.new()
  return _instance
# uso: GameState.get_instance().add_gold(gold_val)
```

Beneficios. Estado de partida único, sin nodo en el árbol (`RefCounted`), 10 usos coherentes. **Desventajas.** Estado global oculto, acopla a los clientes y dificulta pruebas unitarias; el constructor es público (GDScript no tiene constructores privados).

Diagramas de clases



PE-06 · Decorator (estructural)

Qué. Añade responsabilidades a un objeto dinámicamente envolviéndolo. **Participantes → clases.** Component → `DamageSource`; ConcreteComponent → `BaseWeaponDamage`; Decorator → `DamageDecorator`; ConcreteDecorator → `BerserkDamageDecorator`, `UpgradeDamageDecorator`, `StatBlockDamageDecorator`; Client → `Player._damage_chain()`. **Por qué.** Dos funciones casi idénticas repetían la cadena de multiplicadores.

ANTES

```
func _weapon_damage(weapon_type: int) -> float:
  var base_dmg = WeaponData.weapon_damage(weapon_type, is_berserk)
  var mods = get_weapon_modifiers(weapon_type)
  return base_dmg * mods.damage * (stat_block.damage_mult() if stat_block else 1.0)
# _weapon_heavy_damage(): copia con otra base
```

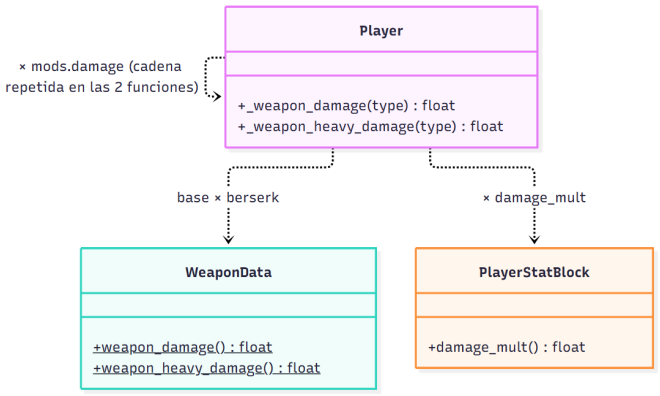
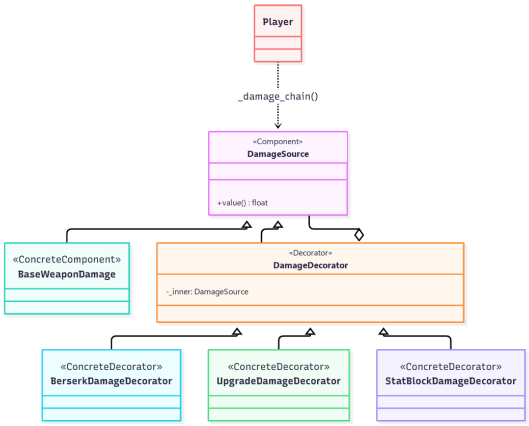
DESPUÉS

```
func _weapon_damage(t: int) -> float: return _damage_chain(t, false).value()
func _weapon_heavy_damage(t: int) -> float: return _damage_chain(t, true).value()

func _damage_chain(t: int, heavy: bool) -> DamageSource:
  var data := get_weapon_data(t)
  var dmg: DamageSource = BaseWeaponDamage.new(data, heavy)
  dmg = BerserkDamageDecorator.new(dmg, data, is_berserk)
  dmg = UpgradeDamageDecorator.new(dmg, get_weapon_modifiers(t))
  dmg = StatBlockDamageDecorator.new(dmg, stat_block)
  return dmg
```

Beneficios. Sin duplicación; modificador nuevo = clase nueva; orden de multiplicación idéntico (probado: 24.8625 = valor esperado). **Desventajas.** Crea 5 objetos `RefCounted` por cálculo de daño (despreciable por ataque, pero no usar dentro de bucles por enemigo y fotograma).

Diagramas de clases

ANTES	DESPUÉS
	

PE-07 · Flyweight (estructural)

Qué. Comparte eficientemente muchos objetos finos separando estado intrínseco (compartido) y extrínseco (por instancia). **Participantes → clases.** Flyweight → `EnemyTypeFlyweight`; FlyweightFactory → `EnemyTypeFlyweightFactory`; Client → `EnemyBehavior.apply_config()`; estado extrínseco → `EnemyData` (posición, hp, cooldowns). **Por qué.** El juego mantiene cientos de `EnemyData`; los datos de cada tipo estaban copiados en 6 clases y se re-asignaban en cada spawn.

ANTES

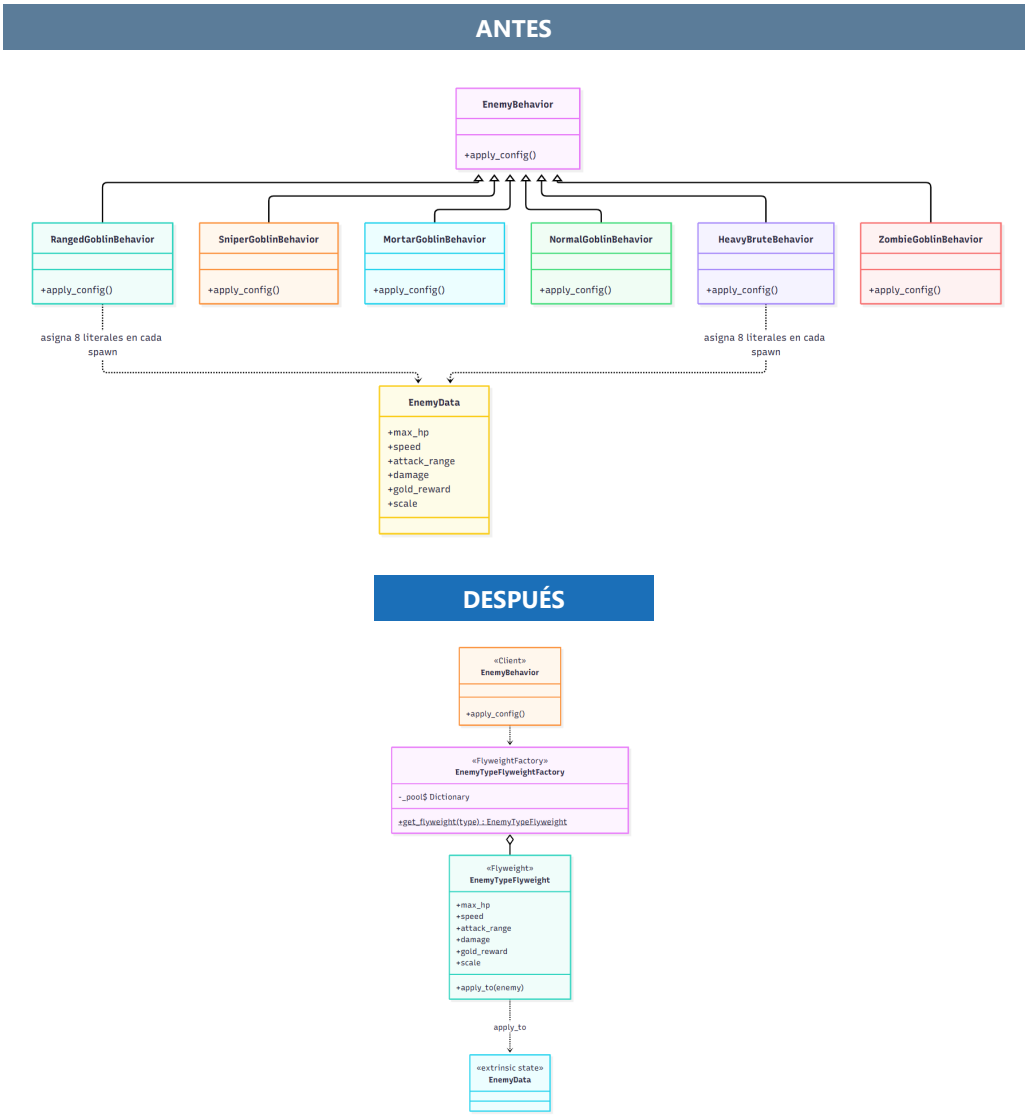
```
class HeavyBruteBehavior extends MeleeGoblinBehavior:
  func apply_config() -> void:
    enemy.max_hp = 90.0
    enemy.hp = 90.0
    enemy.speed = 2.0
    enemy.attack_range = 1.4
    enemy.damage = 0.5
    enemy.gold_reward = 15
    enemy.base_color = Color.WHITE
    enemy.scale = Vector3(1.5, 1.5, 1.5)
```

DESPUÉS

```
  func apply_config() -> void:
    EnemyTypeFlyweightFactory.get_flyweight(EnemyData.EnemyType.HEAVY_BRUTE).apply_to(enemy)
```

Beneficios. 6 flyweights en total para cualquier número de enemigos; balance en un único archivo. **Desventajas.** `apply_to` aún copia valores al enemigo (por rendimiento no se indirecciona cada lectura); el flyweight debe tratarse como inmutable.

Diagramas de clases



PE-08 · Adapter (estructural)

Qué. Convierte la interfaz de una clase en la que espera el cliente. **Participantes → clases.** Target → `MovementSource`; Adapter → `TouchMovementAdapter`, `KeyboardMovementAdapter`, `GamepadMovementAdapter`; Adaptee → autoload `VirtualInput` y API `Input`; Client → `PlayerInputController`. **Por qué.** Un único método mezclaba tres APIs de entrada.

ANTES

```
func get_movement_vector() -> Vector2:
  if VirtualInput.movement_vector.length_squared() > 0.01: return VirtualInput.movement_vector
  ... Input.is_key_pressed(KEY_W) ... Input.get_joy_axis(0, JOY_AXIS_LEFT_X) ...
```

DESPUÉS

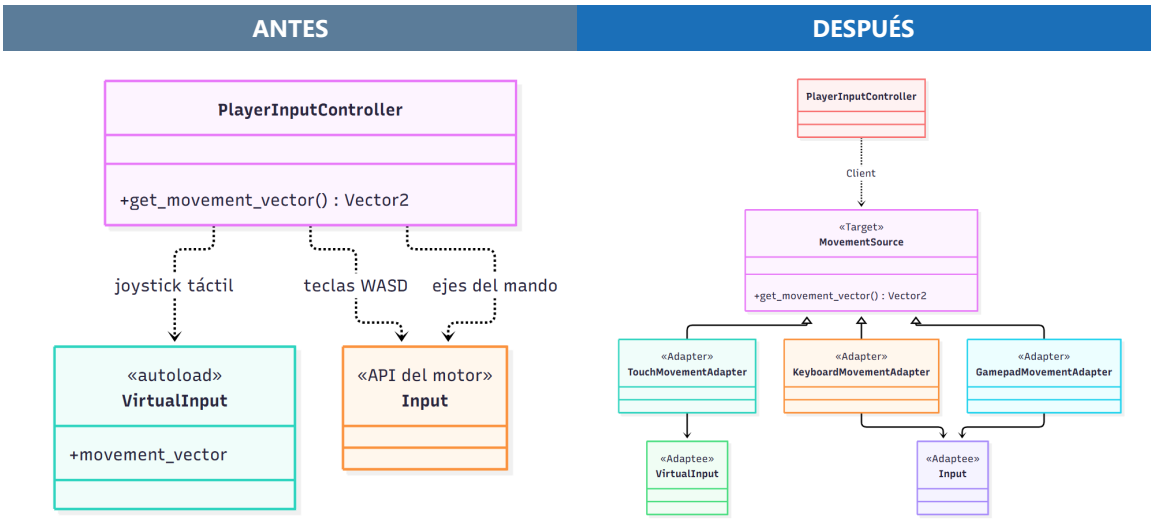
```
var _touch_source: MovementSource = TouchMovementAdapter.new()
var _direct_sources: Array[MovementSource] = [KeyboardMovementAdapter.new(), GamepadMovementAdapter.new()]

func get_movement_vector() -> Vector2:
  var touch := _touch_source.get_movement_vector()
  if touch != Vector2.ZERO: return touch
  var total := Vector2.ZERO
  for source in _direct_sources: total += source.get_movement_vector()
```

```
    return total.limit_length(1.0)
```

Beneficios. El controlador ignora de dónde viene la entrada; nuevo periférico = nuevo adaptador; mismo comportamiento (prioridad táctil, suma teclado+mando, zona muerta 0.2). **Desventajas.** Tres clases pequeñas más; los botones (ataque, habilidad) aún no usan adaptadores.

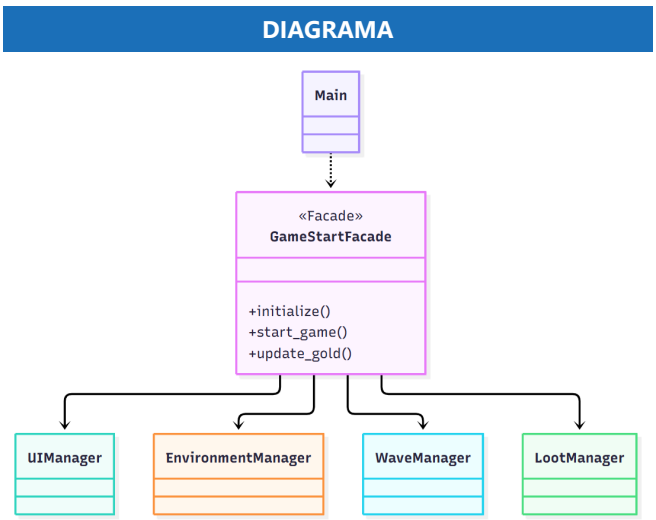
Diagramas de clases



PE-09 · Facade (estructural, existente)

Qué. Interfaz simplificada a un subsistema complejo. **Participantes → clases.** Facade → `GameStartFacade`; subsistemas → `UIManager`, `EnvironmentManager`, `WaveManager`, `LootManager`; Client → `Main`. **Por qué.** `Main` orquestaba más de cuatro subsistemas. Se mantiene de la Entrega 2 y se cuenta como estructural adicional (los 3 mínimos los cubren Decorator, Flyweight y Adapter). **Beneficios.** `Main` solo conoce `initialize()`, `start_game()` y `update_gold()`. **Desventajas.** La fachada crece y puede volverse un "God object" si no se controla.

Diagramas de clases



Ajuste complementario AJ-01 · Object Pool

No es un patrón GoF, pero se corrigió un defecto en `PoolManager`: `get_bullet()` podía reutilizar una bala todavía en vuelo. Ahora busca primero una bala oculta (libre) y solo recicla la más antigua si todas están ocupadas.

Resumen de cobertura frente a la guía

Requisito	Cumplido con
Los 5 creacionales	Factory Method, Abstract Factory, Builder, Prototype, Singleton
Mínimo 3 estructurales	Decorator, Flyweight, Adapter (+ Facade existente)
Estructura de participantes GoF	Tabla de roles en cada sección
Nombres que reflejan el rol	`EnemyCreator`, `WeaponDirector`, `BerserkDamageDecorator`, `MovementSource`…
Código antes/después	Secciones PE-01 a PE-08
Proyecto funcionando	Verificado en Godot 4.7.1 (ver Control de Cambios)